Slide 1:– OpenMP Task Parallelism

Speaker: Ehtesham Faraz

Strategy Overview

Problem

Compute π via numerical integration using a dynamic, load-imbalanced recursive task tree.

Approach

OpenMP task parallelism — one thread seeds the recursion via `#pragma omp single`; all threads steal tasks from the pool.

Recursion

Each task computes π over random steps, then spawns **1–4 child tasks** until `g_num_tasks` is reached.

Atomics

Thread-safe accumulation of `pi_sum` and per-thread counters; unique task IDs via `atomic capture`.

Key Pragmas

`#pragma omp task`

`#pragma omp single`

`#pragma omp atomic`

pi_tasks.c

```
/* Entry point — one thread seeds the whole tree */
#pragma omp parallel
{
    #pragma omp single
    {
        total_tasks_created = 1;
        #pragma omp task firstprivate(seed)
        pi_task(seed);
    }
}

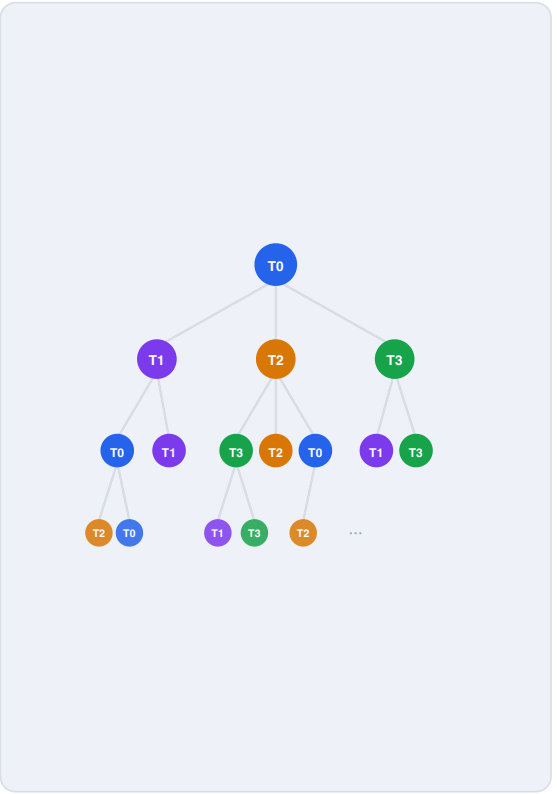
/* Inside pi_task() — runs on any thread */
int tid = omp_get_thread_num();
#pragma omp atomic
tasks_per_thread[tid]++;

double pi = compute_pi(steps);
#pragma omp atomic
pi_sum += pi;

/* Spawn children if budget remains */
for (int i = 0; i < spawn_count; i++) {
    #pragma omp atomic capture
    { my_task_id = total_tasks_created; total_tasks_create
    if (my_task_id < g_num_tasks) {
        #pragma omp task firstprivate(child_seed)
        pi_task(child_seed);
    } else break;
}
```

Task Tree

T0 T1 T2 T3



AVG Π ESTIMATE
3.1416
converges with steps

SPAWN FANOUT
1 – 4
children per task

SYNCHRONISATION
atomic
pi_sum & counters